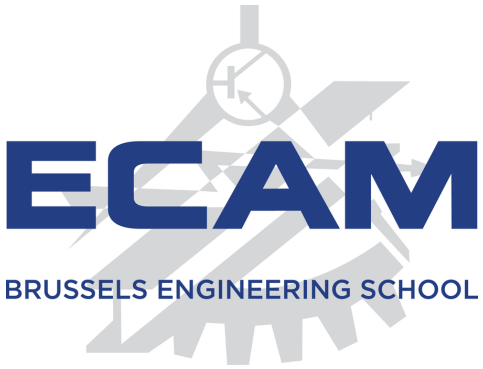


Cahier des charges informatique

CT-CORA3D

Logiciel de planification des ostéotomies de la patte arrière du chien : architecture, technologies, données, sécurité et qualité

Groupe

Dorian Verstappen
Imane Bikass
Mohamed-Amine Ahmidouch

Projet PMD2, Master en ingénierie biomédicale

ECAM Brussels Engineering School, année académique 2026-2027

Version 1.2 du 11 novembre 2026

Table des matières

1 Objet du document **2**

2 Présentation du logiciel **2**

3 Environnement technique **3**

4 Organisation du code **4**

5 Données et formats **4**

6 Plans 2D du guide générique **5**

7 Exigences non fonctionnelles **6**

8 Sécurité **6**

9 Installation et déploiement **7**

9.1 Pour l'utilisateur 7

9.2 Pour le développeur 7

10 Tests et validation **7**

11 Gestion de version et documentation **7**

12 Limites et évolutions **8**

1. Objet du document

Ce document décrit les exigences **informatiques** du logiciel CT-CORA3D : sur quoi il doit fonctionner, comment il est construit, quelles données il lit et produit, comment il est sécurisé et comment sa qualité est vérifiée. Il complète le cahier des charges fonctionnel, qui décrit ce que le logiciel doit faire pour le chirurgien (planifier une ostéotomie du fémur ou du tibia et concevoir les guides chirurgicaux).

2. Présentation du logiciel

CT-CORA3D est une **application de bureau** qui fonctionne entièrement sur l'ordinateur de l'utilisateur, sans connexion internet. Elle comporte deux onglets :

- **Segmentation CT** : lecture du scanner, séparation et identification des os, maillages 3D ;
- **Planification CORA** : repères anatomiques, mesure de la déformation, planification de la correction en 3D, puis construction des guides de coupe et de réduction.

Les calculs lourds (segmentation, guides) sont faits par un **serveur local** écrit en Python. L'interface est une page web (HTML et JavaScript) affichée dans une fenêtre de l'application ; elle dialogue avec le serveur par des requêtes HTTP qui ne quittent jamais l'ordinateur. Le calcul de la planification CORA lui-même s'exécute dans l'interface (JavaScript), ce qui rend la planification interactive.

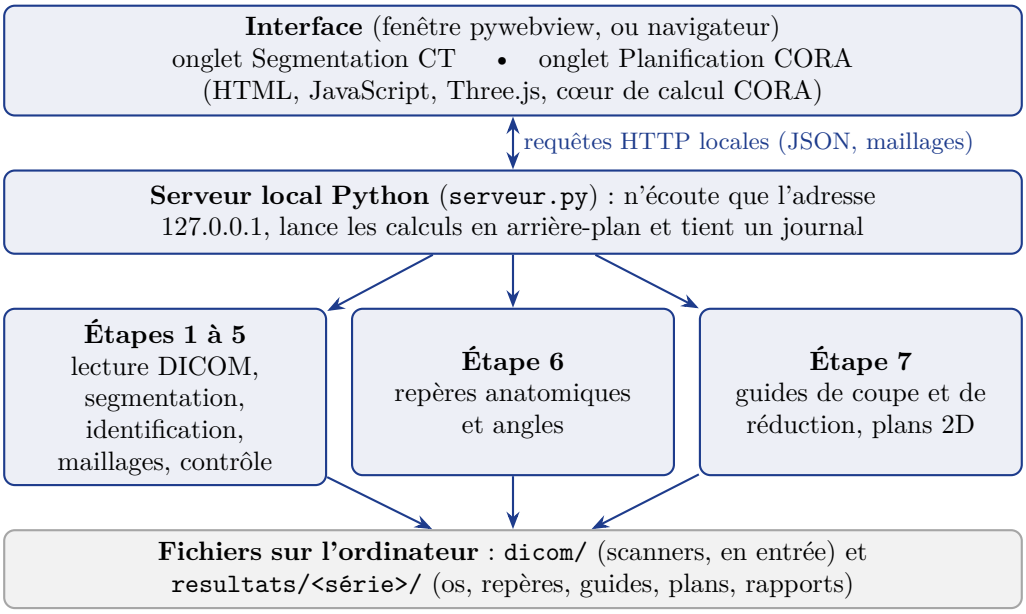


FIGURE 1 : Architecture du logiciel.



FIGURE 2 : Onglet Planification CORA (os fictif : fémur, étape 9 sur 13). À gauche, le cas et la stratégie ; au centre, la vue 3D ; à droite, l'explication de l'étape.

3. Environnement technique

Élément	Choix
Systèmes d'exploitation	macOS et Windows 10 ou 11, avec le même code
Langage des calculs	Python 3.10 ou plus récent
Langages de l'interface	HTML, CSS, JavaScript ; affichage 3D avec Three.js (r170), embarqué dans l'application
Bibliothèques Python	numpy, scipy (calcul) ; pydicom, SimpleITK (lecture DICOM, segmentation) ; vtk, trimesh, pyvista, manifold3d (maillages, opérations booléennes) ; matplotlib, pillow (images, plans 2D)
Plans 2D du guide générique	CADCraft (logiciel de dessin assisté par ordinateur, formats DWG et DXF), piloté par un script Python (section 6)
Fenêtre de l'application	pywebview (installée au premier lancement) ; sinon, le navigateur par défaut
Serveur	module `http.server` de la bibliothèque standard de Python, adresse 127.0.0.1, port libre choisi au démarrage
Application autonome	PyInstaller (environ 800 Mo sur Mac), aucune installation de Python nécessaire
Matériel conseillé	8 Go de mémoire vive au minimum, 16 Go conseillés pour les grands scanners ; 2 Go de disque pour l'application, plus la place des scanners (de quelques centaines de Mo à plusieurs Go par examen) et des résultats

4. Organisation du code

Chaque étape de la chaîne est un fichier Python séparé, que l'on peut aussi lancer seul en ligne de commande.

Fichier ou dossier	Rôle
`application/serveur.py`	serveur local : met les calculs à disposition de l'interface et tient le journal
`application/fenetre.py`, `ct_cora3d_app.py`	fenêtre native de l'application ; point d'entrée de l'application autonome
`application/interface/ct/`	onglet Segmentation CT
`application/interface/cora/index.html`	onglet Planification CORA : un seul fichier, assemblé par `code_source/cora/build_app.py`
`code_source/cora/app/`	cœur de calcul CORA (`cora_simple.js`) et modules de l'interface
`code_source/etape1_lecture_dicom.py` à `etape7_guide.py`	une étape de la chaîne par fichier, de la lecture DICOM aux guides
`code_source/dessin_guide.py`	plan 2D du guide d'un cas, adapté à l'os (conventions ISO)
`documents/plan_guide_generique/`	plans 2D du guide générique (DWG, DXF, PDF) et script qui les dessine dans CADCraft
`code_source/parametres.py`	tous les réglages (seuils, marges, dimensions des guides) au même endroit
`code_source/lancer_tout.py`	étapes 1 à 6 en une seule commande, pour toutes les séries
`code_source/tests/`, `code_source/cora/tests/`	tests automatiques
`application/lanceurs/`	lanceurs Mac et Windows, construction de l'application autonome

5. Données et formats

Entrée. Une série DICOM de scanner par examen, placée dans le dossier `dicom/`. L'utilisateur peut aussi ajouter des points nommés pour aider la segmentation (`graines.json`).

Sorties. Tous les résultats d'une série vont dans `resultats/<série>/` ; ceux d'un guide dans un sous-dossier `guide_<os>_<coin>/`.

Format	Contenu
STL	os segmentés et pièces à imprimer (guide de coupe, guide de réduction)
NRRD, `.mrk.json`	volume de segmentation et repères, lisibles dans 3D Slicer
JSON	bilan d'une série (`rapport.json`), repères et angles d'un os (`cora_<os>.json`), prescription de la correction (`prescription.json`, format v2.2), contrôles d'un guide (`guide.json`)
BIN	maillages pour l'interface : nombre de sommets et de triangles, puis coordonnées (float32) et indices (uint32)
PNG, PDF, SVG	aperçus, images des guides, plans 2D cotés
DWG, DXF	plans 2D du guide générique, modifiables dans CADCraft ou dans un autre logiciel de dessin
Markdown	rapport lisible d'une série (`rapport.md`)

Les longueurs sont en millimètres et les angles en degrés. Chaque os est décrit dans son propre repère (X latéral, Y proximal, Z caudal) ; une matrice 4×4 permet de revenir au repère du scanner. Un membre gauche est traité en miroir, comme un membre droit.

6. Plans 2D du guide générique

Tous les guides sont dérivés d'un même modèle paramétrique, le **guide générique**. Ses deux plans 2D (guide de coupe et guide de réduction) se trouvent dans le cahier des charges fonctionnel. Cette section décrit seulement comment ils sont produits. Ils sont dessinés dans le logiciel de dessin assisté par ordinateur **CADCraft**, mais ils ne sont pas tracés à la main : un script Python reprend les valeurs par défaut du logiciel, calcule la géométrie et envoie les commandes de dessin à CADCraft. Les plans restent ainsi cohérents avec le code et peuvent être redessinés à tout moment.

Élément	Choix
Script	`documents/plan_guide_generique/script/plans_generiques.py` (Python, avec les bibliothèques shapely et pymupdf)
Pilotage	CADCraft est lancé avec son serveur de contrôle, qui n'écoute que l'adresse 127.0.0.1 et exige un jeton ; le script lui envoie des commandes au format JSON (calques, traits, hachures, cotes, textes)
Dessins	deux fichiers séparés, sans l'os : guide de coupe (plan CTC3D-GEN-01) et guide de réduction (plan CTC3D-GEN-02)
Mise en page	feuille A3, échelle 2:1, projection européenne, cartouche, nomenclature et tableau des paramètres
Cotes	vraies cotes du logiciel de dessin (facteur d'échelle 0,5) : dimension réelle, suivie du symbole du paramètre entre parenthèses
Formats	DWG (fichier de travail), DXF (échange avec d'autres logiciels) et PDF (impression à 100 %)
Vérification	relecture du PDF ; barre de contrôle de 50 mm sur chaque plan, pour vérifier l'échelle après impression

7. Exigences non fonctionnelles

N°	Exigence	Critère
NF1	Hors ligne	Le logiciel fonctionne sans connexion internet ; toutes les bibliothèques de l'interface sont embarquées.
NF2	Portabilité	Le même code fonctionne sur macOS et sur Windows, y compris dans des dossiers dont le chemin contient des espaces ou des accents.
NF3	Rapidité	Segmentation d'une série en moins de 2 minutes ; repères en moins de 5 secondes par os ; nouvelle planification en moins de 3 secondes ; guides en moins d'une minute.
NF4	Mémoire	Fonctionnement avec 8 Go de mémoire vive.
NF5	Robustesse	Un os inexploitable ne bloque pas les autres ; chaque erreur est expliquée à l'utilisateur par un message clair.
NF6	Traçabilité	Chaque fichier produit indique le programme, sa version et la date ; chaque calcul est noté dans le journal.
NF7	Reproductibilité	Les mêmes données et les mêmes réglages donnent les mêmes résultats.
NF8	Maintenabilité	Code commenté en français, une étape par fichier, réglages regroupés dans `parametres.py`, documentation à jour.
NF9	Ergonomie	Interface en français, explication de chaque étape, thème clair et sombre.
NF10	Données locales	Les données restent sur l'ordinateur : rien n'est envoyé sur internet (voir section 8).

8. Sécurité

Le logiciel traite des scanners de chiens : ces données ne relèvent pas des normes qui protègent les données médicales humaines, et le logiciel ne les rend donc pas anonymes. La sécurité porte sur le fonctionnement du logiciel :

- Le serveur n'écoute que l'adresse 127.0.0.1 : il n'est pas accessible depuis le réseau.
- Il ne donne accès qu'aux fichiers de l'interface et du dossier des résultats.
- Aucune donnée n'est envoyée sur internet ; il n'y a ni compte ni mot de passe.
- Le dépôt GitHub du projet est public. Le fichier `.gitignore` en exclut le dossier `dicom/`, trop lourd (environ 17 Go), et les résultats des autres scanners ; le dépôt contient les cas de démonstration (os fictif, lévrier).

9. Installation et déploiement

9.1 Pour l'utilisateur

- **Mac** : double-clic sur l'application autonome `CT-CORA3D.app`, ou sur le lanceur, qui crée un environnement Python pour cet ordinateur au premier lancement.
- **Windows** : double-clic sur `Lancer sur Windows.bat`, qui prépare de la même façon son environnement au premier lancement ; l'application autonome Windows se construit sur un PC Windows avec `creer_app_windows.bat`.

9.2 Pour le développeur

1. Cloner le dépôt GitHub (public) et copier ses propres scanners dans `dicom/`.
2. Installer Python 3.10 ou plus récent, puis les bibliothèques de `application/requirements.txt`.
3. Après une modification de l'onglet CORA, réassembler l'interface avec `build_app.py`.
4. Après chaque modification, relancer les tests, puis reconstruire l'application autonome.

10. Tests et validation

Test	Ce qu'il vérifie
`test_chaine.py`	toute la chaîne sur un volume synthétique : lecture, segmentation, noms, maillages étanches, précision de la surface
`test_reperes.py`	repères automatiques sur l'os fictif et sur des os réels (lévrier)
`test_guide.py`	guides : jeu avec l'os, étanchéité, retrait des deux moitiés, guide de réduction
`test_cas.mjs`	planification sur l'os fictif, avec des déformations connues à retrouver
`test_reel.mjs`	planification sur des os segmentés : angles, correction, broches parallèles
`campagne_strategie.mjs`	606 cas : règles chirurgicales (côté du guide, abord, coin recommandé, broches)

Règle du groupe : tous les tests doivent passer avant chaque envoi sur GitHub.

11. Gestion de version et documentation

- **Git et GitHub** : dépôt public, branche principale `main`, un commit par modification cohérente, messages en français.

- **Mac et Windows** : le fichier `.gitattributes` règle les fins de ligne des scripts des deux systèmes.
- **OneDrive** : le dossier du projet est synchronisé ; on évite de travailler sur deux ordinateurs en même temps.
- **Documentation** : `LISEZ-MOI.md` (installation et utilisation), `code_source/README.md` (fonctionnement de chaque étape), `GROUPE.md` (membres du groupe) et le dossier technique en LaTeX.

12. Limites et évolutions

- CT-CORA3D est un **prototype de recherche** : ce n'est pas un dispositif médical certifié, et toute planification doit être validée par un vétérinaire.
- Cette version traite la patte arrière (fémur, tibia) ; l'extension à d'autres os est une évolution possible.
- L'application autonome Windows doit être construite sur un PC Windows.